Introduction

Bit manipulation problems have a reputation for being intimidating, and *Number of Unique XOR Triplets I* is a perfect example of why. On the surface it looks like a counting problem you could brute-force your way through. Underneath, it's testing something much more specific: can you recognize when a property of an operation (in this case, XOR) lets you skip the search entirely?

This is exactly the kind of problem interviewers love, because it separates candidates who pattern-match "loop through everything" from candidates who ask "what does this operation actually guarantee me?" Once you've internalized the trick here, you'll start spotting the same shape — degenerate cases canceling out, then a saturation argument closing the rest — in a huge number of other XOR and bitmask problems.

Problem Overview

You're given a permutation of the integers from `1` to `n` — meaning every number from `1` to `n` appears exactly once, in some shuffled order. You then consider every ordered triplet of indices `(i, j, k)` where `i <= j <= k` (positions can repeat, as long as they're chosen in non-decreasing order), and compute `nums[i] XOR nums[j] XOR nums[k]`.

The question: across every possible triplet, how many **distinct** XOR results can you produce?

Example

Let's use `n = 4`, so our array is some permutation of `[1, 2, 3, 4]` — say `[3, 1, 4, 2]`.

If you exhaustively XORed every valid triplet of positions, you'd find the results cover every integer from `0` to `7` — all 8 possible values in that range. No value in `[0, 7]` is ever skipped, and nothing outside that range ever appears.

Why 8? Because `4` in binary is `100`, which needs 3 bits, and `2³ = 8`. That's the entire answer, and we'll build up to *why* that works.

Intuition

Don't start with the triplets — start with what XOR guarantees you.

Fact 1: XOR-ing a number with itself always produces zero. Every bit matches itself, so every bit cancels. This means if two of your three chosen positions hold the same value, that pair vanishes, and you're left with the third value untouched. So any triplet with a repeat is really just "one original

number in disguise." The only triplets that produce genuinely new results are the ones built from **three distinct numbers**.

Fact 2: Because the array is a permutation of `1..n`, every possible group of three distinct numbers already exists somewhere in it. Position doesn't matter — XOR is commutative and associative, so `a XOR b XOR c` is the same no matter which order you combine them in. This means the array itself is a red herring. The real question is purely about the *set* `{1, 2, ..., n}`: how many distinct values can you get by XOR-ing any three distinct elements together?

Fact 3 (the closer): small sets saturate the whole bit space fast. Take `{1, 2, 3, 4}`. XOR-ing triplets from just these four numbers already produces every integer from `0` to `7` — the full range representable in 3 bits. Once your working numbers span a full bit-width (there's a number with the top bit set, and enough smaller numbers to toggle every bit below it), you can't produce anything new by adding more numbers of that same bit-width. You've already hit every possible combination — this is essentially a pigeonhole argument.

So the entire problem collapses to one question: **how many bits does it take to write `n` in binary?** That's `n.bit_length()` in Python. The answer is `2` raised to that power — with `n = 1` and `n = 2` needing special handling because they're too small to reach full saturation.

Brute Force Solution

Idea: Do exactly what the problem describes — three nested loops over every valid triplet of indices, collecting XOR results in a set.

```
def unique_xor_triplets_brute(nums):
    n = len(nums)
    seen = set()
    for i in range(n):
        for j in range(i, n):
            for k in range(j, n):
                seen.add(nums[i] ^ nums[j] ^ nums[k])
    return len(seen)
```

Advantages: Trivial to write, obviously correct, great for validating the formula against small inputs.

Disadvantages: Falls apart quickly as `n` grows.

Time complexity: O(n³) — three nested loops over the array. **Space complexity:** O(n³) worst case, for the set of results (bounded in practice by the small range of possible XOR values, but the *work* to get there is still cubic).

Optimal Solution

Once you accept the two facts above — repeats collapse to single values, and any three distinct numbers from `1..n` are available regardless of the array's order — the problem becomes: **what's the size of the XOR-closure of `{1, ..., n}` under 3-way combination?**

The closure size is governed entirely by the bit-width of `n`:

n	bit_length(n)	Unique XOR values
1	—	1 (special case)
2	—	2 (special case)
3	2	4
4	3	8
7	3	8
8	4	16

For `n >= 3`, the answer is `1 << n.bit_length()`, i.e., `2 ** n.bit_length()`.

Python Code

```
def unique_xor_triplets(n: int) -> int:
    """Count distinct XOR results over all triplets from a permutation of 1..n."""
    if n == 1:
        return 1
    if n == 2:
        return 2
    return 1 << n.bit_length()
```

Code Walkthrough

- `if n == 1: return 1` — with only one element, every triplet is `(1, 1, 1)`, XOR-ing to `1`. Only one possible result.
- `if n == 2: return 2` — too small to saturate a full bit-width; the only distinct results are `1` and `2`.
- `1 << n.bit_length()` — `n.bit_length()` gives the number of bits needed to represent `n`. Shifting `1` left by that many places computes `2 ** n.bit_length()` without calling `pow`, using a single fast bitwise operation.

Dry Run

Take `n = 4`. `n.bit_length()` is `3` (binary `100` needs 3 bits). `1 << 3 = 8`. The function returns `8`, matching the brute-force result over `[1, 2, 3, 4]`.

Complexity Analysis

Optimal solution: O(1) time and O(1) space — `bit_length()` and a bit shift are constant-time operations regardless of how large `n` gets.

Why it's correct: The saturation argument guarantees that once a set of consecutive integers starting at 1 spans a full bit-width, three-way XOR combinations of that set produce every value in `[0, 2^bit_length - 1]`, and adding more same-width numbers can't produce values outside that range (XOR of b-bit numbers is always a b-bit number) or introduce gaps (the smaller numbers already provide enough freedom to toggle every bit).

Alternative Solutions

If the bit-math derivation feels shaky, a legitimate fallback is to brute-force only a small prefix of the array (say, the first 10–15 elements) and stop once the set of seen results stops growing. It's easy to write and easy to trust without deriving the closed form, but it's an empirical pattern-match rather than a proven guarantee — use it as a stepping stone, not a final answer in an interview.

Edge Cases

- `n = 1`: only one triplet possible, answer is `1`.
- `n = 2`: too small for saturation, answer is `2`.
- `n = 3`: first case where the full formula applies, answer is `4`.
- Very large `n` (e.g., `100000`): formula still runs in constant time.
- Any permutation ordering: the answer is arrangement-invariant, since it depends only on the *set* of values, not their positions.

Common Mistakes

1. Forgetting that positions can repeat (`i <= j <= k`, not strictly increasing), which is what enables the cancellation argument in the first place.
2. Assuming XOR triplet count depends on the specific permutation rather than just the values `1..n`.
3. Missing the `n = 1` and `n = 2` special cases and applying the formula everywhere.
4. Confusing `bit_length()` with the value of `n` itself (e.g., using `n` instead of `1 << n.bit_length()`).
5. Attempting to brute-force at scale (`n = 100000`) and hitting a timeout instead of recognizing the closed-form pattern.

Interview Questions

- Can you prove why three numbers from `1..4` cover all values from `0` to `7`?

- What changes if the array isn't a permutation but has duplicates?
- How would you extend this to 4-way XOR triplets instead of 3?
- Why does XOR's self-cancellation property matter more here than, say, addition would?

Similar Problems

- **Single Number** (LeetCode) — same self-cancellation property of XOR, used to find a unique element.
- **Maximum XOR of Two Numbers in an Array** — another problem built entirely around bit-length reasoning.
- **Total Hamming Distance** — bit-by-bit thinking across an array.
- **Counting Bits** — builds intuition for how bit-length scales with value.

Key Takeaways

Repeated indices in a triplet cancel out under XOR, so only truly distinct triplets matter. Because the array is a permutation, position is irrelevant — the answer depends only on the set `{1, ..., n}`. And small sets saturate the full bit-width space almost immediately, collapsing the entire problem into `2 ** n.bit_length()`, with two small exceptions at `n = 1` and `n = 2`.

Watch the Video

For the full walkthrough with visuals of the bit-cancellation and saturation argument, watch the video here: {LINK}

About the Series

This problem is part of the **Daily Python LeetCode** series, where each day's official LeetCode challenge gets solved from first principles in Python — starting from the obvious brute force and working toward the insight that makes it fast, with a beginner-friendly explanation the whole way through.

Call To Action

If this helped the bit-length trick click, like the video on YouTube, subscribe for tomorrow's problem, and subscribe to the Substack so you never miss a daily writeup. Drop a comment if you want the brute-force fallback covered in more depth, and share this with anyone prepping for coding interviews.

Quick Review

What pattern applies to 'Number of Unique XOR Triplets I' (n≤3)?

Small-n brute-force / bit trick: with only 1-3 elements, enumerate all XOR combinations directly instead of searching for a clever formula.

Time complexity for counting unique XOR triplets when n≤3?

O(n^3) worst case, but bounded by constant since n≤3, so effectively O(1) per test but O(n^3) in general form.

Space complexity of the unique-XOR-triplet counting approach?

O(n^3) in the worst case to store all triplet XOR results in a set, though with small n it's negligible.

Common bug when counting 'unique' XOR results?

Forgetting to dedupe with a set/hash before counting, so repeated XOR values get counted multiple times instead of once.

XOR triplet brute force vs. two-pointer/prefix-XOR pattern — when does each apply?

Brute force fits tiny fixed n (≤3); prefix-XOR with hashing scales better when n grows large and pairwise/triplet counts are needed.

Interview follow-up: how would you solve this if n were up to 10^5?

Precompute prefix XORs and use a hash map to count combinations achieving each XOR value, avoiding the O(n^3) enumeration.
